LABORATORY OBSERVATION / RECORD

Course Full Stack Web Development
Course Code 23CM4121
Experiment No. 6
Student Name P. Govindamma
Roll No. A24126552104
Date 29-08-2026

1. TITLE

Develop a Mini Application using Express and Node.js (Task Tracker with REST API)

2. AIM

To develop a mini web application using Node.js and the Express framework that exposes a REST API for creating, reading, updating and deleting tasks, and to serve a simple front-end page that uses the API.

3. OBJECTIVE

The objectives of this experiment are:

- To set up a Node.js project and install Express using npm.
- To create an Express server and define routes for the HTTP methods GET, POST, PUT and DELETE.
- To use built-in middleware (express.json() and express.static()).
- To validate input and return proper HTTP status codes (200, 201, 400, 404).
- To build a simple HTML page that calls the API using fetch().
- To test each route and verify the output.

4. THEORY

Node.js is a JavaScript runtime that runs JavaScript outside the browser. **Express** is a minimal third-party web framework for Node.js, installed with npm and loaded with require('express'). It simplifies creating an HTTP server, defining routes and using middleware.

Routing maps an HTTP method and a URL path to a handler function, for example app.get('/tasks', handler).

Middleware are functions that run between the request and the response. express.json() parses JSON request bodies into req.body. express.static('public') serves files from the public folder.

REST API: A REST API uses HTTP methods to work on resources. In this experiment the resource is a task: GET reads tasks, POST creates a task, PUT updates a task and DELETE removes a task. Responses are sent as JSON with status codes such as 200 (OK), 201 (Created), 400 (Bad Request) and 404 (Not Found).

The tasks are stored in an in-memory array, so the data resets whenever the server restarts.

5. ALGORITHM / PROCEDURE

- Create a new project folder and initialize it using npm init.
- Install Express using npm install express.
- Create package.json with a start script that runs node server.js.
- Create server.js, import Express using require() and create the app.
- Add the middleware express.json() and express.static('public').
- Create an array tasks with two sample tasks.
- Define the home route GET / that sends public/index.html.
- Define GET /tasks to return all tasks as JSON.
- Define POST /tasks to validate the title, create a task with a new id and return status 201.
- Define PUT /tasks/:id to find the task and toggle its completed value, or return 404.
- Define DELETE /tasks/:id to remove the task, or return 404.
- Start the server with app.listen() on port 3000.
- Create public/index.html that calls the API using fetch().
- Run npm start, test every route and verify the actual output against the expected output.

6. PROGRAM

package.json

```
{
  "name": "task-tracker",
  "version": "1.0.0",
  "description": "Task Tracker using Node.js and Express.js",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

server.js

```
const express = require('express');
const app = express();

app.use(express.json());
app.use(express.static('public'));

let tasks = [
    { id: 1, title: 'Complete assignment', completed: false },
    { id: 2, title: 'Study Node.js', completed: false }
];

// Home route
```

```
app.get('/', (req, res) => {
    res.sendFile('index.html', { root: 'public' });
});

// Get all tasks
app.get('/tasks', (req, res) => {
    res.json(tasks);
});

// Add a task
app.post('/tasks', (req, res) => {
    const { title } = req.body;

    if (!title || !title.trim()) {
        return res.status(400).json({
            message: 'Task title is required'
        });
    }

    const task = {
        id: tasks.length
            ? Math.max(...tasks.map(t => t.id)) + 1
            : 1,
        title: title.trim(),
        completed: false
    };

    tasks.push(task);
    res.status(201).json(task);
});

// Mark task as completed or incomplete
app.put('/tasks/:id', (req, res) => {
    const task = tasks.find(
        t => t.id === Number(req.params.id)
    );

    if (!task) {
        return res.status(404).json({
            message: 'Task not found'
        });
    }

    task.completed = !task.completed;
    res.json(task);
});

// Delete a task
app.delete('/tasks/:id', (req, res) => {
    const id = Number(req.params.id);
    const exists = tasks.some(t => t.id === id);

    if (!exists) {
        return res.status(404).json({
            message: 'Task not found'
        });
    }

    tasks = tasks.filter(t => t.id !== id);
    res.json({ message: 'Task deleted successfully' });
});
```

```
// Start server
app.listen(3000, () => {
    console.log('Task tracker running at http://localhost:3000');
});
```

public/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Task Tracker</title>
</head>
<body>
  <h1>Task Tracker</h1>

  <input id="title" placeholder="New task">
  <button onclick="addTask()">Add</button>

  <ul id="list"></ul>

  <script>
    async function loadTasks() {
      const res = await fetch("/tasks");
      const tasks = await res.json();

      const list = document.getElementById("list");
      list.innerHTML = "";

      tasks.forEach(function (t) {
        const li = document.createElement("li");
        li.innerHTML =
          '<input type="checkbox" ' + (t.completed ? "checked" : "") +
          ' onchange="toggleTask(' + t.id + ')"> ' +
          t.title +
          ' <button onclick="deleteTask(' + t.id + ')">Delete</button>';
        list.appendChild(li);
      });
    }

    async function addTask() {
      const input = document.getElementById("title");
      await fetch("/tasks", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ title: input.value })
      });
      input.value = "";
      loadTasks();
    }

    async function toggleTask(id) {
      await fetch("/tasks/" + id, { method: "PUT" });
      loadTasks();
    }

    async function deleteTask(id) {
      await fetch("/tasks/" + id, { method: "DELETE" });
      loadTasks();
    }

    loadTasks();
  </script>
```

```
</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
require('express')	Imports the Express module installed through npm.
express()	Creates an Express application instance.
app.use(express.json())	Middleware that parses JSON request bodies into req.body.
app.use(express.static('public'))	Middleware that serves static files from the public folder.
let tasks = [...]	In-memory array that stores the tasks.
app.get('/')	Sends index.html for the home page.
app.get('/tasks')	Returns all tasks as JSON.
app.post('/tasks')	Validates the title, creates a task with a new id and returns status 201.
app.put('/tasks/:id')	Finds the task by id and toggles its completed value; returns 404 if not found.
app.delete('/tasks/:id')	Removes the task with the given id; returns 404 if not found.
req.params.id	Reads the :id value from the URL.
res.status(n).json({...})	Sets the HTTP status code and sends a JSON response.
app.listen(3000)	Starts the server on port 3000.
fetch() in index.html	Browser API used to call the server routes.

8. TEST CASES AND EXECUTION DATA

Each route was executed and the actual output was recorded against the expected output.

Test Case	Route	Action	Expected Result	Actual Result	Status
TC-01	server.js	Run npm start	Server starts on port 3000	Task tracker running at http://localhost:3000	Pass
TC-02	GET /	Open http://localhost:3000/	Home page displayed	HTML page returned (200)	Pass
TC-03	GET /tasks	Request all tasks	Two sample tasks returned	Two tasks returned (200)	Pass
TC-04	POST /tasks	Add "Read Express docs"	New task with id 3 created	Task created (201)	Pass
TC-05	POST /tasks	Send an empty title	Validation error	Task title is required (400)	Pass
TC-06	PUT /tasks/1	Toggle task 1	completed becomes true	completed: true (200)	Pass
TC-07	PUT /tasks/99	Toggle a missing task	Not found error	Task not found (404)	Pass
TC-08	DELETE /tasks/2	Delete task 2	Task removed	Task deleted successfully (200)	Pass
TC-09	DELETE /tasks/99	Delete a missing task	Not found error	Task not found (404)	Pass

9. OUTPUT

Each route was tested with curl while the server was running. The terminal output for each test case is given below.

Output 1 (TC-01): Server start

```
$ npm start
Task tracker running at http://localhost:3000
```

Output 2 (TC-02): GET /

The browser shows the Task Tracker page served from public/index.html. (Paste your browser screenshot here.)

Output 3 (TC-03): GET /tasks

```
$ curl http://localhost:3000/tasks
[{"id":1,"title":"Complete assignment","completed":false},
 {"id":2,"title":"Study Node.js","completed":false}]
HTTP 200
```

Output 4 (TC-04): POST /tasks

```
$ curl -X POST -H "Content-Type: application/json" \
  -d '{"title":"Read Express docs"}' http://localhost:3000/tasks
{"id":3,"title":"Read Express docs","completed":false}
HTTP 201
```

Output 5 (TC-05): POST /tasks with empty title

```
$ curl -X POST -H "Content-Type: application/json" \
  -d '{"title":"  "}' http://localhost:3000/tasks
{"message":"Task title is required"}
HTTP 400
```

Output 6 (TC-06): PUT /tasks/1

```
$ curl -X PUT http://localhost:3000/tasks/1
{"id":1,"title":"Complete assignment","completed":true}
HTTP 200
```

Output 7 (TC-07): PUT /tasks/99

```
$ curl -X PUT http://localhost:3000/tasks/99
{"message":"Task not found"}
HTTP 404
```

Output 8 (TC-08): DELETE /tasks/2

```
$ curl -X DELETE http://localhost:3000/tasks/2
{"message":"Task deleted successfully"}
HTTP 200
```

Output 9 (TC-09): DELETE /tasks/99

```
$ curl -X DELETE http://localhost:3000/tasks/99
{"message":"Task not found"}
HTTP 404
```

10. RESULT

Thus, a mini Task Tracker application was successfully developed using Node.js and Express. The server defined routes for GET, POST, PUT and DELETE, used the express.json() and express.static() middleware, validated input and returned proper HTTP status codes. A simple HTML page was served from the public folder and used fetch() to call the API. All test cases were executed and passed.